WORKSHOP 1 · SEMANAS 1–6

Spin Network Laboratory

Simulador dinámico de redes topológicas en C++17 y SFML 2.6

> Sistema computacional inspirado en redes de espín para construir, visualizar, modificar y analizar grafos topológicos dinámicos mediante estructuras de datos personalizadas, integración numérica e invariantes combinatorios.

Problema y propósito

Una red contiene vértices y relaciones. Sus operaciones deben mantener una representación consistente al insertar, conectar, eliminar o redimensionar el almacenamiento. El proyecto conecta esas operaciones con un dibujo dinámico: cada edición produce un cambio visible y sus efectos pueden inspeccionarse.

El objetivo académico es demostrar estructuras propias con gestión clara de memoria, no sólo utilizar un contenedor ya implementado. La tabla hash almacena los vértices; las listas simplemente enlazadas almacenan sus conexiones. Una aplicación SFML procesa entrada, presenta el grafo y permite experimentar con fuerzas artificiales.

Alcance científico

“QuantumVertex” es un nombre conceptual para un vértice etiquetado. Los espines son etiquetas semienteras positivas. No se imponen reglas de acoplamiento SU(2), ni se modelan amplitudes cuánticas. Las fuerzas organizan la visualización y no representan la dinámica real de gravedad cuántica.

Entrega	Resultado
Workshop 1 · semanas 1–6	Grafo editable, física y renderizado SFML.
Workshop 2 · semanas 7–12	Historial C++, invariantes, refinamiento y UI final.

Equipo: completar con los integrantes reales. Repositorio GitHub: URL no proporcionada. Documento preparado el 6 de octubre de 2026.

Una propiedad explícita

La aplicación de escritorio separa almacenamiento, dinámica y presentación. El núcleo y el controlador no incluyen SFML: sus pruebas se ejecutan sin crear una ventana.

```
DesktopApplication ── InputController / GraphRenderer / HUD
        │
        └── SimulationController
                ├── QuantumGraph
                │     └── HashTable<uint64_t, QuantumVertex>
                │             └── QuantumVertex
                │                   └── SinglyLinkedList<SpinEdge>
                └── ForceEngine / VerletIntegrator
```

Propietarios y observadores

Los nodos de la tabla y de la lista poseen su enlace siguiente mediante `std::unique_ptr`. La tabla posee cada valor y cada bucket; el vértice posee su lista de adyacencia. La cola de la lista es un puntero observador: no libera memoria. No se utilizan `new` ni `delete` manuales para gestionar nodos.

Una arista contiene `targetId` y parámetros, no un puntero al vértice destino. Rehash puede reconstruir los vértices en otras direcciones; las conexiones siguen siendo válidas porque se resuelven mediante IDs. El controlador también conserva IDs para selección, conexión y arrastre.

Tiempo de vida

`findVertex` devuelve un observador. Su validez termina al eliminar ese vértice, reconstruir la tabla o asignar otro grafo. El motor de fuerzas utiliza observadores únicamente durante una evaluación sin cambios topológicos. Ninguno se almacena entre frames.

La limpieza recorre y desmonta las cadenas iterativamente; evita una destrucción recursiva proporcional a la longitud. Las copias de la lista y la tabla son profundas: cada copia obtiene nodos internos propios. Los objetos movidos se pueden reutilizar.

Organización del código

Carpeta	Responsabilidad
include/DataStructures	Lista simple y hash propia.
include/Core y Physics	Modelo, vectores, fuerzas e integrador.
include/Application y Graphics	Controlador, ventana, entrada y HUD.
src / tests / assets / docs	Implementaciones SFML, pruebas, fuente tipográfica e informe.

El uso de STL como infraestructura —buckets, buffers y ordenamiento— no reemplaza las estructuras enlazadas y hash que se evalúan.

Adyacencias sin ownership ambiguo

`SinglyLinkedList<T>` implementa la colección de conexiones de cada vértice. Cada nodo contiene un valor y un `unique_ptr` al siguiente. La lista mantiene cabeza propietaria, cola observadora y contador de elementos.

```
head (unique_ptr)
  └─ [valor | next] ── [valor | next] ── [valor | null]
                                              ↑
                                         tail (observador)
```

Operación	Contrato / costo
pushFront(value)	Inserta en cabeza; actualiza cola si estaba vacía. O(1).
pushBack(value)	Enlaza desde cola y actualiza contador. O(1).
find / contains	Recorren por igualdad; find devuelve observador o null. O(n).
remove / removeIf	Eliminan la primera coincidencia y reparan cola. O(n).
clear / size	Limpieza iterativa O(n), consulta O(1).
begin / end	Iteración mutable o constante sin exponer los nodos.

Eliminación y cola

La eliminación modifica el enlace propietario que apunta al nodo: puede ser la cabeza o el `next` del anterior. El siguiente nodo se mueve a ese enlace. Si el nodo eliminado era la cola, la cola pasa al anterior; si era el único elemento, queda nula.

La variante `removeIf` permite eliminar una arista por identificador de destino sin exigir igualdad completa entre todos sus parámetros. La búsqueda por valor requiere un tipo T comparable sólo cuando se llama a esas operaciones.

Copia y movimiento

El constructor de copia crea una lista temporal y copia valores con `pushBack`. Al completar, intercambia su contenido. La asignación por valor e intercambio evita aliasing entre cadenas. El movimiento transfiere ownership y deja la fuente vacía y reutilizable.

Pruebas del avance

Se verifican inserción al frente y al final, búsqueda constante y mutable, ausencia, eliminación de cabeza, interior y cola, reinserción después de vaciar, iteradores y copia independiente. Se construye además una cadena de 10.000 elementos, se mueve, se reutiliza el origen y se limpia.

> La cola nunca es propietaria. La memoria pertenece a la cadena de `unique_ptr` iniciada en la cabeza.

Búsqueda por ID con hash propia

`HashTable<uint64_t, QuantumVertex>` usa encadenamiento separado. El arreglo de buckets pertenece a la tabla; cada bucket inicia una cadena propia de nodos clave/valor. Identificadores duplicados se rechazan sin sobrescribir el valor original.

FNV-1a de 64 bits

$$h_0 = 14695981039346656037$$

$$h_{i+1} = (h_i \text{ XOR } b_i) \times 1099511628211 \pmod{2^{64}}$$

$$\text{index(key)} = \text{hash(key)} \bmod \text{capacity}$$

Las claves enteras se serializan en bytes little-endian. Así el resultado no depende del endianness del procesador. Una prueba verifica que FNV-1a de “hello” produce `0xa430d84680aabd0b`. También se admiten cadenas e inyección de un hash alternativo para probar colisiones.

Encadenamiento

```
bucket[0] → [clave, valor] → [clave, valor] → null
bucket[1] → null
bucket[2] → [clave, valor] → null
```

Insertar, buscar y eliminar recorren únicamente la cadena correspondiente. El costo esperado es O(1) con distribución adecuada; una cadena patológica puede requerir O(n). La prueba de colisiones usa una función constante: 1.000 claves distintas caen deliberadamente en el mismo bucket.

Factor de carga y reconstrucción

$$\alpha = \text{elementos} / \text{buckets} \quad \text{objetivo: } \alpha \le 0.75$$

Antes de insertar una clave que supere el umbral, se amplía la tabla. Rehash construye una tabla temporal y copia los valores. Sólo después de terminar intercambia tablas. Si una asignación de memoria falla, la tabla original no cambia.

Esta garantía tiene costo: copiar un vértice incluye copiar sus adyacencias. Rehash es O(V+E) esperado para este grafo; con hashes patológicos, las búsquedas de duplicados en las cadenas pueden llevarlo a $O(V^2)$. No se conserva un observador durante esta operación.

Evidencia

Se insertan 5.000 vértices, se buscan todos, se rechazan duplicados, se fuerza rehash a 16.384 buckets y se eliminan todas las entradas. La prueba de colisiones incluye borrado de cabeza, interior y cola y reutilización de tablas copiadas/movidas.

Una arista lógica, dos adyacencias

`QuantumGraph` representa un grafo simple no dirigido. Al conectar a y b crea un registro a→b y otro b→a, con el mismo ID lógico y los mismos parámetros. El contador global aumenta una sola vez.

QuantumVertex	SpinEdge
ID, masa, volumen	ID de arista y targetId
Posición y posición anterior	Espín positivo semientero
Velocidad, aceleración, fuerza	Longitud natural y rigidez
pinned y lista de conexiones	Destino resuelto por ID

Validaciones

Se rechazan vértices con posición no finita o masa no positiva. La sobrecarga que recibe un vértice completo exige además estado físico válido y adyacencias vacías: las conexiones sólo se crean por la API del grafo. Las aristas rechazan parámetros inválidos, lazos, duplicados y extremos ausentes.

Si falla la creación de la segunda adyacencia por falta de memoria, se retira la primera antes de propagar la excepción. El contador no se incrementa hasta completar ambas. Esto mantiene el contrato no dirigido.

Eliminar un vértice

Primero se obtiene una lista de IDs vecinos. Luego se elimina cada arista recíproca y se reduce el contador lógico. Finalmente se elimina el vértice de la tabla. Los demás vértices no conservan referencias a un ID que desapareció.

API pública

```
addVertex / removeVertex / findVertex / containsVertex
addEdge / removeEdge / hasEdge
vertexCount / numberOfEdges / degree / clear
```

Las listas de adyacencia se modifican únicamente mediante la API del grafo, nunca directamente desde un observador mutable.

Pruebas

Un grafo de 1.000 vértices y 5.000 aristas verifica simetría y conteo. Cada nodo tiene grado 10. Al eliminar el vértice 0 se retiran sus 10 aristas; ningún vecino conserva una conexión a él. Copiar y modificar el original no afecta la copia. Reinsertar el ID produce un vértice aislado.

> Los invariantes básicos ya existentes se conservan como complemento. El panel analítico y la integración con historial corresponden al Workshop 2.

Geometría dinámica, tiempo fijo

Para vértices i y j, $\Delta = x_j - x_i$ y $r = |\Delta|$. Las fuerzas de visualización se evalúan por pares con signos opuestos. No se confunden con interacción cuántica.

$$F^s_{ij} = k_{global} \cdot k_{arista} \cdot (r - L) \cdot \Delta/(r+\varepsilon_s)$$
$$F^r_{ij} = -k_r \cdot \text{dirección} / (r^2+\varepsilon^2)$$
$$F^d_i = -\gamma v_i \qquad a_i = F_i/m_i$$

La longitud natural L es 90 en los presets. La repulsión usa $\varepsilon=5$; el pequeño ε_s del resorte evita división por cero. Si dos nodos coinciden, una dirección determinista basada en IDs proporciona una fuerza de separación. Se evita introducir infinitos para fijar un vértice.

Velocity Verlet

1. Evaluar fuerzas conservativas y calcular a_n.
2. Aplicar medio amortiguamiento: $v \leftarrow \exp(-\gamma\Delta t/(2m))v$.
3. Medio impulso: $v \leftarrow v + (\Delta t/2)a_n$.
4. Avanzar posición: $x_{n+1} \leftarrow x_n + \Delta t \cdot v$.
5. Recalcular fuerzas en x_{n+1} y obtener a_{n+1}.
6. Completar impulso y amortiguamiento: $v \leftarrow \exp(-\gamma\Delta t/(2m))[v + (\Delta t/2)a_{n+1}]$.

El amortiguamiento se separa simétricamente porque depende de velocidad. El caso conservativo conserva aproximadamente energía; el amortiguado la disipa. Un nodo fijo o arrastrado omite integración de posición y mantiene velocidad cero.

Independencia de renderizado

$$\Delta t = 1/120 \text{ s} \qquad \text{física: 120 pasos/s}$$

El renderizado aporta tiempo al acumulador. Se ejecutan pasos completos hasta consumirlo. Una pausa borra el acumulador. Una suspensión larga se limita a 0.1 s de recuperación, como máximo 12 pasos, para evitar una espiral de actualización.

Pruebas numéricas

Un oscilador de dos nodos mantiene error relativo de energía inferior al 0.1% tras 1.200 pasos sin amortiguamiento. Con amortiguamiento la energía baja; una masa libre cumple $v(t)=v(0)\exp(-\gamma t/m)$. El estado tras un segundo coincide a 30, 60 y 144 FPS. Cien nodos permanecen finitos durante 7.200 pasos.

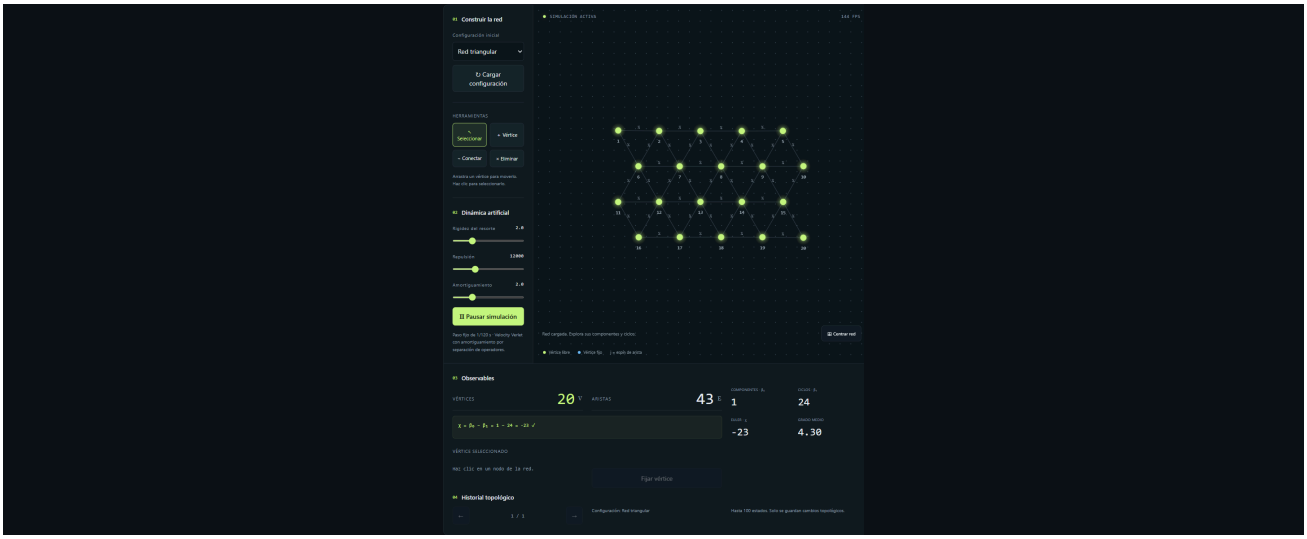
El cálculo de fuerzas cuesta $O(V^2+E)$ esperado por paso. El avance usa cálculo directo; no se promete rendimiento para millones de nodos.

Manipular y observar

La aplicación SFML dibuja vértices como círculos, aristas como segmentos e IDs junto a los nodos. El HUD incluye pausa, cantidad de vértices/aristas y parámetros de resortes, repulsión y amortiguamiento.

Acción	Control
Agregar / seleccionar / conectar	A / S / C, o botones y clics.
Eliminar seleccionado / por puntero	Delete / D y clic.
Pausar / fijar / reiniciar posiciones	Space / P / R.
Repulsión / rigidez / amortiguamiento	Up–Down / [–] / ,–.
20 nodos / 100 nodos / encuadrar	1 / 2 / F.

El arrastre conserva el punto de agarre: seleccionar un borde del nodo no lo hace saltar al centro del puntero. Reiniciar posiciones conserva topología y fijaciones. Encuadrar modifica sólo la cámara. La ventana se puede cerrar con Esc o el control del sistema.



Captura real de la demo web en Brave. Es JavaScript independiente y no acredita ejecución de SFML.

Estado de comprobación

La aplicación SFML compila con GCC y CMake. Este entorno restringido bloquea la conexión X11: aún no se afirma haber observado su ventana. El comando `./bin/spin_network --smoke-test` está preparado para abrir 100 nodos, renderizar, guardar captura, probar pausa/repulsión y cerrar en un escritorio habilitado.

El controlador ya tiene pruebas automáticas de edición, arrastre, fijación, pausa y eliminación. El guion de `docs/validation.md` completa la comprobación manual. La demo web se conserva como complemento independiente y sus controles sí se probaron en Brave.

Evidencia y cierre de alcance

Comprobación	Resultado
Lista simple y memoria propietaria	Pruebas pasan; nodos con unique_ptr.
Hash: 5.000 vértices, colisiones, rehash	Pasa; 1.000 claves forzadas a un bucket.
Grafo y eliminación incidente	Pasa; 1.000 nodos / 5.000 aristas.
Física y 100 nodos estables	Pasa; 7.200 pasos, 60 s simulados.
30 / 60 / 144 FPS	Mismo estado tras un segundo.
CMake / CTest	Aplicación compilada; 3/3 suites pasan.
ASan + UBSan	Tres suites pasan, detect_leaks=0.
Ventana SFML / fugas / Windows	Pendientes de validación externa.

El benchmark de física optimizada dio aproximadamente 0.19–0.23 ms por paso para 100 nodos en este equipo. No mide tiempo de renderizado, ni establece una garantía para otros equipos.

Limitaciones concretas

LeakSanitizer aborta bajo ptrace en este entorno. ASan y UBSan no reportaron errores con detección de fugas desactivada; no se afirma ausencia de fugas verificada. La conexión X11 devuelve Operation not permitted, por lo que la aceptación visual sigue abierta. Los binarios generados son Linux x86_64; Windows/Code::Blocks requiere compilación y prueba con SFML compatible.

Cómo reproducir

```
./tools/build.sh
./bin/spin_network
./bin/spin_network --smoke-test
cmake -S . -B build && cmake --build build
ctest --test-dir build --output-on-failure
```

Conservar assets/fonts y usar SFML 2.6.x. El README explica dependencias locales y Code::Blocks. Los fuentes incluyen tests, scripts de compilación, el informe y la demo web.

Workshop 2

La entrega final añadirá lista doble y estados con copia profunda, undo/redo, eliminación de la rama futura, límite de 100 snapshots topológicos, invariantes y panel analítico. El refinamiento local u–v → u–w–v preservará χ porque ΔV=ΔE=1. Multithreading es opcional; un movimiento Pachner 3D queda fuera del alcance obligatorio.

Referencias: SFML 2.6 — Linux; API SFML 2.6.2; implementación y evidencias en include/, tests/ y docs/validation.md.